

Edocs — Backend Implementation Log

What was built in `edocs_backend`, the decisions behind it, how it was verified, what had to be fixed along the way, and what is still open.

Scope of this log	Spring Boot backend, DevOps files, documentation and one small frontend change (OAuth2 redirect)
Inputs used	<code>edocs_class_diagram_v4.png</code> · <code>Edocs_Project_Document.docx</code> · <code>web-technology-project-instruction.pdf</code> · <code>edocs_frontend/src</code> (<code>api.ts</code> , <code>types.ts</code> , <code>rbac.ts</code> , <code>mock.ts</code>)
Stack	Java 21 · Spring Boot 4.1.1 · Hibernate 7.4 · Jackson 3.1 · Spring AMQP 4.1 · PostgreSQL 17 · MongoDB 8 · RabbitMQ 4
Status	Implemented, tested, verified end-to-end; NOT committed (left for the author to commit)
Date	3 October 2026

Contents

1. Summary	3
2. What was built, module by module	3
3. Key decisions	5
4. Verification performed	5
5. Problems found during verification and how they were fixed	7
6. Known gaps and honest limitations	7
7. Suggested commits (to be made by the author)	8
8. How to run	8

1. Summary

- **114** production Java classes in 11 modules, **12** test classes, one Flyway migration (15 tables).
- Implements **every endpoint** the frontend's `services/api.ts` calls, with identical JSON shapes, plus `/auth/me` and OAuth2 login.
- PostgreSQL for the transactional core, MongoDB for encrypted content, audit chain and notifications, RabbitMQ for email/SMS/events.
- **47 automated tests executed, 47 passed** (9 Testcontainers twins skipped locally because Docker could not start). The real React app was driven in headless Chrome against the real API: 121 API calls, all 2xx, no console errors.
- Comment style follows the request: single-line `//` comments only where they add meaning (106 across ~6,900 lines, no block or multi-line comments).

2. What was built, module by module

Package	Contents	Key behaviour
identity	User, Person, Organization, Membership, Role, AuthService, MemberService, SettingsService + controllers	Login with lockout and rate limit, MFA challenge, OAuth2 user mapping, invitations with temporary password, self-MFA toggle, settings
security	SecurityConfig, JwtService, JwtAuthUserConverter, PrincipalService, RolePermissions, TokenDenylist, OAuth2LoginSuccessHandler, LoginRateLimiter	HS256 JWT resource server, live role/status per request, RBAC authorities, logout revocation, JSON 401/403
document	ContentItem, Document, Contract, Party, Template, DocumentShare, DocumentContent (Mongo), ContentStore, DocumentService, ArchiveService, TemplateService, DocumentAccess	Create from template, versioned encrypted saves, comments, shares, send/remind, delete drafts only, archive search, legal hold, metrics
signing	Signature, OtpChallenge, OtpService, SigningService, SignatureArtifact, AnchorListener	OTP intent, strict signing order, evidence hash, seal + Merkle root, async anchoring
workflow	Workflow, WorkflowStep, RoutingRule, WorkflowService, ReviewEscalationJob	Four-step contract workflow, evidence package, routing rules (CFO routing and escalation active)
audit	AuditLog (Mongo), AuditService, AuditController	Hash-chained append with optimistic sequencing, verification, scoped listing
notification / messaging	Notification (Mongo), NotificationService, RabbitConfig, MessagePublisher, NotificationListeners, SmsGateway	Bell + email/SMS via RabbitMQ after commit, DLQ, delivery status
compliance / dashboard	ComplianceControl, ComplianceService, DashboardService	Controls evaluated from live data; KPIs, risk bands, bottlenecks
common / config / seed	ApiException handler, Hashing, EnvelopeCrypto, HtmlSanitizer, TimeFormat, CacheConfig, OpenApiConfig, DemoDataSeeder	Uniform errors, crypto, caching, OpenAPI, the same demo workspace as the frontend mock

Other files added

File	Purpose
edocs_backend/pom.xml, mvnw, .mvn/	Build (Spring Boot 4.1.1 parent, JaCoCo), Maven wrapper for CI
src/main/resources/application.yml, application-dev.yml	12-factor configuration; dev profile enables demo MFA code and OTP echo
src/main/resources/db/migration/V1__init_schema.sql	Full relational schema with PK/FK/CHECK/UNIQUE/indexes

File	Purpose
edocs_backend/Dockerfile, .dockerignore, .gitignore, README.md	Container image, run and API reference
edocs_frontend/Dockerfile, nginx.conf, .dockerignore	Production SPA image
docker-compose.yml (repo root)	Full stack: PostgreSQL, MongoDB, RabbitMQ, Mailpit, API, frontend
.github/workflows/ci.yml	CI: backend verify, frontend build, images to GHCR on main
documentation/*.pdf, diagrams/, screenshots/	This log, the project report, diagrams and responsive screenshots

Frontend change (small, needed for OAuth2)

services/api.ts gained completeOAuthRedirect(): after Google/Microsoft sign-in the backend redirects to /login#token=...; the SPA reads the token from the fragment, loads /auth/me and stores the session. state/auth.tsx calls it on start, and LoginPage.tsx shows ?error= messages. TypeScript still compiles cleanly.

3. Key decisions

Decision	Alternative considered	Reason
Layered modular monolith + RabbitMQ events	Microservices from day one	Project document recommends phase-1 monolith; modules are extraction-ready
Flyway migration + Hibernate validate	JPA ddl-auto=update	Hand-written CHECKs, partial indexes, RESTRICT actions and a reviewable history; Hibernate cannot generate these
UUID primary keys	Slugs like the mock (q3-master-agreement)	Matches the class diagram, non-guessable, no collisions across tenants
JWT resource server (HS256) + OAuth2 login	Server sessions	Stateless API scales horizontally; OAuth2 handled by Spring Security
Re-read role/status per request (30 s cache)	Trust role claim in JWT	Role changes and deactivation apply immediately, as the frontend expects
Encrypt bodies per document	Plain text in DB	Confidentiality at rest; key wrapping enables crypto-shredding (GDPR erasure)
Strict signing order and party check	Any user with document:sign may sign	Non-repudiation: only the named signer, in order, can sign
Audit list scoped by permission	403 for non-auditors	Dashboard calls it for every role; non-auditors see only their own actions
Publish after commit	Publish inside transaction	No emails for rolled-back work

4. Verification performed

4.1 Automated tests

```
$ mvn test
Tests run: 9   com.edocs.security.RolePermissionsTest      pass
Tests run: 5   com.edocs.audit.AuditServiceTest          pass
Tests run: 5   com.edocs.common.EnvelopeCryptoTest       pass
Tests run: 4   com.edocs.common.HashingAndSanitizerTest  pass
Tests run: 7   com.edocs.signing.SigningRulesTest        pass
Tests run: 5   com.edocs.compliance.ComplianceEvaluationTest pass
Tests run: 3   com.edocs.document.ArchiveSearchTermsTest pass
Tests run: 9   com.edocs.EmbeddedFlowTest (69 s)         pass
Tests run: 9   com.edocs.ContainersFlowTest              skipped (no Docker)
Tests run: 56, Failures: 0, Errors: 0, Skipped: 9   BUILD SUCCESS
JaCoCo line coverage: 1439 / 1772 = 81.2%
```

The nine end-to-end journeys: 401 without token · uniform wrong-password message · admin MFA (wrong then right code) · RBAC per role and signer visibility · signer gets 404 on others' documents · full lifecycle draft → save (XSS stripped) → comment → send → wrong OTP 422 → OTP → sign → sealed, edit/delete 409, anchor event via RabbitMQ, chain verify intact, evidence package · logout revokes token · deactivated member loses access immediately · dashboard and compliance.

4.2 Live server smoke test

```
CORS preflight from http://localhost:5173 -> 200, Allow-Origin echoed, credentials true
CORS preflight from http://evil.example -> 403
GET /documents, /dashboard, /audit-logs, /templates, /settings, /members, ... -> 200 in 20-70 ms
Content-Encoding: gzip; X-Frame-Options: DENY; Referrer-Policy: no-referrer; nosniff
GET /api/v3/api-docs -> 200
```

4.3 Real frontend against real backend

The React app (Vite, VITE_API_BASE_URL=http://localhost:8080/api) was driven with Playwright in headless Chrome at 1440, 820 and 390 px: sign-in, dashboard, documents, editor, workflow, archive, templates, compliance, settings, and a signer session. Result: **121 API calls, 0 non-2xx, 0 console or page**

errors; the signer saw only the Q3 contract. Screenshots are in documentation/screenshots.

5. Problems found during verification and how they were fixed

Symptom	Root cause	Fix
App refused to start: wrong column type for sha256	SQL CHAR(64) vs entity VARCHAR (Hibernate validate)	Columns changed to VARCHAR(64) with a regex CHECK for 64 hex characters (stricter)
Queues not declared on the embedded broker	Apache Qpid rejects RabbitMQ x-dead-letter-* arguments	Dead-lettering made configurable (on by default; off only in the Docker-free harness)
/actuator/health returned 503	Mail health check failed without SMTP	Mail health disabled: email is async with retries and must not fail readiness
Health still 503 in harness	mongo-java-server lacks the hello command	Mongo health disabled only in the embedded test harness
Signer PUT /settings returned 400 not 403	Body validation runs before @PreAuthorize	Test sends a valid body; documented behaviour
Docker unavailable	WSL is not installed, so Docker Desktop's engine cannot start	Added Docker-free stack (embedded PostgreSQL, Mongo-wire server, Qpid) for tests and local runs
Ports 5173 and 4173 busy	Another dev/preview server already running	Left them untouched; ran the verification frontend on port 3000

6. Known gaps and honest limitations

- **Not verified locally:** the Testcontainers suite (needs Docker/WSL) and real Google/Microsoft sign-in (needs client IDs/secrets). Both are wired and configured; CI will run the container suite.
- **Simulated, by design:** blockchain anchoring (Merkle root recorded in the audit chain, no public chain), the qualified trust service provider certificate, and the SMS carrier (LoggingSmsGateway).
- **Single-instance assumptions:** the logout deny-list and login rate limiter are in memory. For several API replicas, move them to Redis.
- **Cross-store consistency:** MongoDB writes are not part of the PostgreSQL transaction. A rare failure between the two can leave an orphan content document. An outbox or compensating job would close this.
- **Search:** free-text archive search decrypts candidate bodies in memory (fine at demo scale). Large tenants need an encrypted search index.
- Invitations email a temporary password; a set-password link flow would be better UX. Two routing rules (fast-track NDA, parallel KYC) are stored but have no automated effect yet.
- The frontend has no vitest dependency installed, so CI type-checks and builds it but does not run its unit tests.

7. Suggested commits (to be made by the author)

Nothing was committed. Suggested sequence on a new backend branch, run from the repository root:

#	git add ...	Commit message
1	edocs_backend/pom.xml edocs_backend/mvnw edocs_backend/mvnw.cmd edocs_backend/.mvn edocs_backend/.gitignore edocs_backend/src/main/java/com/edocs/EdocsApplication.java edocs_backend/src/main/resources/application*.yaml	build(backend): scaffold Spring Boot 4.1 project with Maven wrapper and config
2	edocs_backend/src/main/resources/db edocs_backend/src/main/java/com/edocs/common edocs_backend/src/main/java/com/edocs/config	feat(backend): add Flyway schema, error handling, envelope encryption and caching
3	edocs_backend/src/main/java/com/edocs/identity edocs_backend/src/main/java/com/edocs/security	feat(auth): JWT resource server, OAuth2 login, MFA, lockout and RBAC
4	edocs_backend/src/main/java/com/edocs/audit	feat(audit): hash-chained audit log in MongoDB with verification
5	edocs_backend/src/main/java/com/edocs/notification edocs_backend/src/main/java/com/edocs/messaging	feat(messaging): RabbitMQ email/SMS notifications and domain events
6	edocs_backend/src/main/java/com/edocs/document edocs_backend/src/main/java/com/edocs/workflow	feat(documents): contract lifecycle, templates, archive and workflows
7	edocs_backend/src/main/java/com/edocs/signing	feat(signing): OTP intent, ordered signatures, sealing and anchoring
8	edocs_backend/src/main/java/com/edocs/compliance edocs_backend/src/main/java/com/edocs/dashboard edocs_backend/src/main/java/com/edocs/seed	feat(backend): compliance checks, dashboard KPIs and demo data seeder
9	edocs_backend/src/test	test(backend): unit tests and end-to-end journeys (Testcontainers + embedded)
10	edocs_backend/Dockerfile edocs_backend/.dockerignore edocs_backend/README.md edocs_frontend/Dockerfile edocs_frontend/nginx.conf edocs_frontend/.dockerignore docker-compose.yml .github	ci: Docker images, compose stack and GitHub Actions pipeline
11	edocs_frontend/src/services/api.ts edocs_frontend/src/state/auth.tsx edocs_frontend/src/pages/LoginPage.tsx	feat(frontend): complete OAuth2 sign-in from backend redirect
12	documentation	docs: project report, implementation log, diagrams and screenshots

Then push and open the pull request:

```
git push -u origin backend
gh pr create --base main --head backend --title "Spring Boot backend for Edocs" \
  --body "Implements the REST API the frontend calls (PostgreSQL + MongoDB, RabbitMQ, OAuth2/JWT, RBAC), Docker/CI and docs."
```

8. How to run

```
# Whole stack (Docker Desktop with WSL 2)
docker compose up --build

# Backend without Docker (embedded PostgreSQL, Mongo-wire server, AMQP broker)
cd edocs_backend
./mvnw spring-boot:test-run -Dspring-boot.run.main-class=com.edocs.LocalDevApplication

# Frontend against it
cd edocs_frontend
VITE_API_BASE_URL=http://localhost:8080/api npm run dev

# Demo accounts: password Demo@2026 (admin MFA code 246810 in the dev profile)
j.davis@acme.corp (admin) · s.jenkins@acme.corp (legal) · m.vance@partnercorp.io (signer) · e.rostova@acme.corp (auditor)
```